

004_ARCHITECTURE.md

ЛомХаб — Архитектура проекта

Версия: 1.0

Общая архитектура

Тип приложения:

Full Stack Web Application

Архитектура:

- Frontend — Next.js
 - Backend — Next.js Route Handlers
 - База данных — PostgreSQL
 - ORM — Prisma
 - Авторизация — JWT + HttpOnly Cookies
 - Карты — Яндекс Карты
 - Хранилище файлов — S3-совместимое Object Storage
 - Контейнеризация — Docker
-

Структура проекта

```
lomhub/  
  
app/  
api/  
  (auth)/  
  dashboard/  
  map/  
  prices/  
  yards/  
  
components/  
ui/  
layout/  
forms/  
map/  
yard/  
dashboard/
```

```
lib/  
auth/  
db/  
maps/  
utils/  
validators/  
  
prisma/  
schema.prisma  
seed.ts  
  
public/  
  
styles/  
  
types/  
  
hooks/  
  
services/  
  
docs/
```

Принципы разработки

1. TypeScript без использования `any`.
2. Компоненты должны быть переиспользуемыми.
3. Логика не хранится внутри UI-компонентов.
4. Все запросы проходят через сервисный слой.
5. Все данные валидируются на сервере.
6. Все страницы адаптивны.

API

REST API первой версии.

Основные маршруты:

```
GET    /api/cities  
GET    /api/yards  
GET    /api/yards/{id}  
  
GET    /api/metals  
  
POST   /api/auth/login
```

```
POST    /api/auth/register
POST    /api/auth/logout

GET      /api/dashboard

PUT      /api/prices

POST     /api/reviews
```

Prisma

Все изменения базы данных выполняются только через Prisma.

Запрещается:

- писать SQL прямо в коде приложения;
- дублировать модели.

Все изменения проходят через миграции.

Авторизация

Используется:

JWT

HttpOnly Cookie

Роли:

- user
- yard_owner
- admin

Каждый запрос проверяет права доступа.

Работа с файлами

Фотографии приемок:

- загружаются через API;
- автоматически оптимизируются;
- сохраняются в Object Storage.

Хранятся только ссылки на файлы.

Работа с картой

Используются Яндекс Карты.

Для каждой приемки хранятся:

- широта;
- долгота.

Маркеры загружаются динамически.

Логирование

Все ошибки записываются.

Логируются:

- ошибки API;
 - ошибки авторизации;
 - ошибки загрузки файлов.
-

Конфигурация

Все секреты находятся только в переменных окружения.

Пример:

```
DATABASE_URL
JWT_SECRET
YANDEX_MAPS_API_KEY
S3_ENDPOINT
S3_ACCESS_KEY
S3_SECRET_KEY
```

Docker

Проект запускается одной командой.

```
docker compose up
```

Контейнеры:

- web
- postgres

На MVP этого достаточно.

Git

Основные ветки:

```
main  
develop
```

Новые функции создаются в отдельных ветках:

```
feature/map  
feature/dashboard  
feature/reviews  
feature/auth
```

Код-стайл

Имена компонентов:

PascalCase

Пример:

```
PriceTable.tsx  
MapFilters.tsx  
YardCard.tsx
```

Функции:

camelCase

Константы:

Производительность

Цели MVP:

- Lighthouse ≥ 90
 - First Contentful Paint ≤ 2 сек
 - Mobile First
 - Server Side Rendering для SEO-страниц
-

Масштабирование

Архитектура должна позволять без изменения существующих моделей добавить:

- мобильное приложение;
- биржу заявок;
- уведомления;
- API для партнеров;
- аналитику;
- систему лояльности.

Все новые функции должны быть расширением существующей архитектуры, а не ее заменой.